36.4 共享组件

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~190k tokens，输出 ~5k tokens（本节）

macOS 和 iOS 应用之间存在大量共享逻辑——Gateway WebSocket 协议、聊天 UI、Canvas 命令解析、设备认证等。这些共享代码统一放在 `apps/shared/OpenClawKit/` 中。

36.4.1 OpenClawKit 架构

Swift 包结构

OpenClawKit 是一个 Swift 包，包含三个库 target：

```
// apps/shared/OpenClawKit/Package.swift
let package = Package(
    name: "OpenClawKit",
    platforms: [.iOS(.v18), .macOS(.v15)],
    products: [
        .library(name: "OpenClawProtocol", targets: ["OpenClawProtocol"]),
        .library(name: "OpenClawKit", targets: ["OpenClawKit"]),
        .library(name: "OpenClawChatUI", targets: ["OpenClawChatUI"]),
    ],
    dependencies: [
        .package(url: "ElevenLabsKit", exact: "0.1.0"),   // TTS 语音合成
        .package(url: "textual", exact: "0.3.1"),          // Markdown 渲染
    ],
)
```

三层架构

```
┌──────────────────────┐
│   OpenClawChatUI     │  ← SwiftUI 聊天组件（macOS + iOS 共用）
│   依赖: OpenClawKit   │
│   依赖: Textual       │  ← Markdown 渲染
├──────────────────────┤
│   OpenClawKit        │  ← 核心业务逻辑
│   依赖: OpenClawProtocol│
│   依赖: ElevenLabsKit │  ← TTS 提供商
├──────────────────────┤
│   OpenClawProtocol   │  ← 纯数据模型 + 协议定义
│   无外部依赖          │
└──────────────────────┘
```

层	职责	文件数
OpenClawProtocol	Gateway JSON 模型（`GatewayModels`）、Wizard 帮助函数、AnyCodable	3
OpenClawKit	WebSocket 客户端、设备认证、Canvas 命令、TTS、能力声明、Deep Links	39
OpenClawChatUI	SwiftUI 聊天视图、消息渲染、Markdown、会话管理	13

OpenClawProtocol — 纯数据层

最底层只包含 Gateway 通信的数据模型，没有任何外部依赖：

```
Sources/OpenClawProtocol/
├── AnyCodable.swift        # 动态 JSON 值类型
├── GatewayModels.swift     # Gateway 帧、Hello、事件等类型
└── WizardHelpers.swift     # 引导向导的辅助类型
```

这种分层确保了协议类型可以被任何 target 引用，而不会引入不必要的依赖。

OpenClawKit — 核心逻辑

中间层包含与 Gateway 交互的所有核心逻辑：

```
Sources/OpenClawKit/
├── GatewayChannel.swift          # WebSocket 客户端 (protocol + actor)
├── GatewayNodeSession.swift      # 节点会话管理
├── GatewayTLSPinning.swift       # TLS 证书钉扎
├── GatewayPush.swift             # 推送通知集成
├── DeviceIdentity.swift          # Ed25519 设备密钥
├── DeviceAuthStore.swift         # 设备 token 存储
├── Capabilities.swift            # 能力声明枚举
├── CanvasCommands.swift          # Canvas 操控命令
├── CanvasA2UIAction.swift        # Agent-to-UI 动作
├── CanvasA2UICommands.swift      # A2UI 命令定义
├── CanvasCommandParams.swift     # Canvas 参数类型
├── CameraCommands.swift          # 摄像头命令
├── ScreenCommands.swift          # 屏幕录制命令
├── LocationCommands.swift        # 位置命令
├── SystemCommands.swift          # 系统命令
├── TalkDirective.swift           # Talk Mode 指令
├── TalkPromptBuilder.swift       # Talk 提示词构建
├── TalkSystemSpeechSynthesizer.swift  # 系统 TTS
├── TalkHistoryTimestamp.swift    # 对话历史时间戳
├── BonjourTypes.swift            # Bonjour 服务发现类型
├── BonjourEscapes.swift          # Bonjour 名称转义
├── DeepLinks.swift               # 深度链接处理
├── StoragePaths.swift            # 文件路径常量
├── JPEGTranscoder.swift          # JPEG 编解码
└── ...
```

WebSocket 客户端

`GatewayChannel` 是共享的 WebSocket 客户端，通过协议抽象（`WebSocketTasking`）与具体的网络实现解耦：

```swift
// apps/shared/OpenClawKit/Sources/OpenClawKit/GatewayChannel.swift
public protocol WebSocketTasking: AnyObject {
    var state: URLSessionTask.State { get }
    func resume()
    func cancel(with closeCode: URLSessionWebSocketTask.CloseCode, reason: Data?)
    func send(_ message: URLSessionWebSocketTask.Message) async throws
    func receive() async throws -> URLSessionWebSocketTask.Message
}

extension URLSessionWebSocketTask: WebSocketTasking {}
```

`URLSession` 的默认实现设置了 16MB 的最大消息大小——大型快照和历史 payload 可能超过默认限制：

```swift
extension URLSession: WebSocketSessioning {
    public func makeWebSocketTask(url: URL) -> WebSocketTaskBox {
        let task = self.webSocketTask(with: url)
        task.maximumMessageSize = 16 * 1024 * 1024  // 16 MB
        return WebSocketTaskBox(task: task)
    }
}
```

连接选项通过 `GatewayConnectOptions` 结构体传递：

```swift
public struct GatewayConnectOptions: Sendable {
    public var role: String          // "node" 或 "operator"
    public var scopes: [String]      // 权限范围
    public var caps: [String]        // 设备能力
    public var commands: [String]    // 支持的命令
    public var permissions: [String: Bool]  // 已获取的权限
    public var clientId: String
    public var clientMode: String
}
```

能力系统

设备能力通过一个简洁的枚举定义：

```swift
public enum OpenClawCapability: String, Codable, Sendable {
    case canvas       // 画布操控
    case camera       // 摄像头
    case screen       // 屏幕录制
    case voiceWake    // 语音唤醒
    case location     // 位置信息
}
```

每个能力对应一组命令类型（如 `CameraCommands` 、 `ScreenCommands` ），Gateway 根据节点声明的能力列表来决定暴露哪些工具给 AI Agent。

OpenClawChatUI — 共享聊天界面

最上层是 SwiftUI 聊天组件，被 macOS 和 iOS 共同使用：

```
Sources/OpenClawChatUI/
├── ChatView.swift                  # 主聊天视图
├── ChatViewModel.swift             # 聊天视图模型
├── ChatComposer.swift              # 消息输入组件
├── ChatMessageViews.swift          # 消息气泡渲染
├── ChatModels.swift                # 聊天数据模型
├── ChatTransport.swift             # 聊天传输协议
├── ChatPayloadDecoding.swift       # 消息 payload 解码
├── ChatSessions.swift              # 会话管理
├── ChatSheets.swift                # 聊天相关的 Sheet 视图
├── ChatTheme.swift                 # 聊天主题配色
├── ChatMarkdownPreprocessor.swift # Markdown 预处理
├── ChatMarkdownRenderer.swift      # Markdown 渲染（使用 Textual）
└── AssistantTextParser.swift       # 助手文本解析
```

`ChatView` 是一个完整的 SwiftUI 视图，包含消息列表、输入框、会话切换等。它通过 `ChatTransport` 协议与具体的网络层解耦——macOS 和 iOS 各自提供不同的 transport 实现。

Android 的共享策略

Android 应用无法直接使用 Swift 包。它的共享策略是：

1. **资源文件复用**：通过 Gradle 的 `assets.srcDir` 直接引用 `OpenClawKit/Resources`
2. **协议重新实现**：在 `ai.openclaw.android.protocol` 包中用 Kotlin 重新定义了能力枚举、命令类型等
3. **通信协议一致**：虽然语言不同，但 JSON 格式的 WebSocket 帧是跨平台统一的

本节小结

1. **OpenClawKit 是 macOS 和 iOS 共享的 Swift 包**，采用三层架构：Protocol（纯数据）→ Kit（核心逻辑）→ ChatUI（SwiftUI 组件）。
2. **WebSocket 客户端**通过协议抽象与 URLSession 解耦，支持 16MB 大消息和 TLS 证书钉扎。
3. **能力系统**通过 5 个枚举值声明设备硬件能力，Gateway 据此决定可用工具。
4. **ChatUI 层**提供完整的 SwiftUI 聊天界面，通过 `ChatTransport` 协议支持不同的网络后端。
5. **Android 通过资源共享和协议重实现**来保持跨平台一致性，JSON 帧格式是三个平台的统一契约。